FORMAL-VERIFICATION AUDIT

COINjecture Whitepaper v2.6 — Lean 4 Proof Claims vs. Repository Artifacts

DOCUMENT	DARQ-LV-001 v0.2.0 (Working Draft)	**DATE**	30 May 2026
SUBJECT	COINjecture Network LLC — whitepaper v2.6 (LaTeX re-typeset)	**CLASSIFICATION**	Internal — pre-publication review
PREPARED BY	A. Zalewski, DARQ Labs LLC	**ANALYSIS**	Source inspection + hole sweep (see §2, §7)
SOURCE	`COINjecture-Network/COINjecture2.0` @ `005fcf1` (2026-05-30)	**TOOLCHAIN**	`lean4 v4.28.0` / `mathlib4 v4.28.0`

Revision v0.2.0 — re-points section references to the v2.6 LaTeX re-typeset (appendices A–D are now top-level sections). Findings, tier verdicts, and line citations are unchanged. The re-typeset removed one overclaim sentence ('Each inequality below is Lean-verified') and introduced a duplicate 'References' heading; neither affects the verdicts.

1 Executive summary

The Lean 4 corpus cited by the whitepaper is genuine, but it is materially narrower than the paper's prose implies. Across **25 audited claim-points**, 8 are backed by kernel-checkable proofs carrying full logical content (Tier I) — notably the uniqueness of the eigenvalue *mu*, the reward-monotonicity result, and the first-block emission value. Three rest on floating-point `native_decide` checks evaluated at sample points (Tier II). Eight are declared axioms — several of type `: True`, which is logically inert — and so prove nothing (Tier III). Six are unbacked by any artifact or are contradicted by the repository (Tier IV).

The Tier-IV set is the consequential one: the `(q/p)^z` magnitude bound (only the trivial inequality `q^z < p^z` is proven), the asymptotic-emission claims, the "Factorization NP-hard" table entry (incorrect on the merits), the Private-Submissions zero-knowledge description (the code ships a testnet SHA-256 MAC placeholder, not ZK), and the stated toolchain version. **Crucially, the Lean files themselves carry an honest Tier-A / Tier-B labeling that the whitepaper does not propagate — the gap is one of representation, not fabrication.** Two items (the ZK description and the toolchain version) should be corrected before any external circulation.

TIER	MEANING	COUNT
I — proof	Kernel-checkable proof over the naturals / reals / complex field; full logical content.	8
II — numeric	Decided by compiled evaluation (`native_decide`) at specific Float sample points.	3
III — axiom	Declared axiom, cited not reproved; where typed `: True` it is logically inert.	8
IV — unbacked	No corresponding artifact, or the repository state contradicts the paper.	6

2 Scope & method

The four files cited by whitepaper references [6]–[9] were read in full, together with their immediate dependencies in the `lean4/Coinjecture/` tree: `ClassicalAxioms`, `Coherence`, `ComplexDecomposition`, `DesignAxioms`, `Rewards`, `SecurityCalculations`, `SecurityModel`, `WorkScore`, and `Falsifiability`. Each whitepaper assertion was matched to the artifact named in the reference (or to the absence of one), and classified by the *kind* of Lean construct backing it: a kernel proof, a compiled numeric check, a declared axiom, or nothing. Every cell in the matrix that follows carries a `file:line` citation verifiable against the commit named above.

Classification is by source inspection plus a mechanical hole sweep for `sorry`, `admit`, `axiom`, and `native_decide`. The Lean kernel was **not** re-executed in this environment (a full Mathlib rebuild was not run). A Tier-I designation therefore means the proof uses legitimate, kernel-checkable tactics and carries full logical content — *contingent on the zero-error build the paper reports*. See §7 for the formal limits of this method.

3 Epistemic tier legend

I — proof	**Machine-checked.** A theorem proven by kernel-checkable tactics over `Nat`, `Real`, or `Complex` (core Lean or Mathlib). Carries full logical content.
II — numeric	**Numeric check.** Discharged by `native_decide` on `Float` at specific sample points. Establishes IEEE-754 rounding behaviour, not a universally-quantified real-analytic theorem; adds the Lean compiler and `Lean.ofReduceBool` to the trusted base.
III — axiom	**Declared axiom.** Assumed, not proven. Where typed `: True`, the declaration is logically inert — it neither constrains the model nor serves as a non-trivial hypothesis; it is a named placeholder / TODO.
IV — unbacked	**Unbacked or contradicted.** No corresponding Lean artifact exists, or the repository state contradicts the paper's description.

4 Claim-by-claim verification matrix

#	PAPER CLAIM (section)	LEAN EVIDENCE (file : line)	WHAT IS ACTUALLY ESTABLISHED	TIER
A · EIGENVALUE mu, DESIGN AXIOMS & COHERENCE				
1	Symmetry + unit-normalization give |x| = |y| = 1/sqrt(2). *(Design Axioms)*	`DesignAxioms.lean:39` unit_circle_eta; `:43` symmetry_equal_magnitudes `(native_decide)`	Float evaluations of 2·eta^2 = 1 and |Re| = Im at eta = 1/sqrt(2) decide as expected at the machine level; the analytic step 2x^2 = 1 => x = 1/sqrt(2) is not itself formalized.	II — numeric
2	|mu| = 1 (unit-modulus / unitarity); mu lies on the unit circle.	`ComplexDecomposition.lean:185` mu_abs (over Complex); duplicate `DesignAxioms.lean:41` mu_on_unit_circle `(native_decide)`	|mu| = 1 is a genuine kernel-checkable proof over the complex field; the DesignAxioms twin is a Float numeric check only.	I — proof
3	mu = (-1 + i)/sqrt(2) is uniquely selected by Re < 0. *(mu_unique)*	`ComplexDecomposition.lean:210` mu_unique; `:183` def mu	The three constraints (unit modulus, -Re = Im, Re < 0) are proven to determine mu uniquely over the complex field. Fully earned.	I — proof
4	Anchor · coherence factorization; anchors lie on the light cone.	`ComplexDecomposition.lean:41` Anchor.abs_value; `:79` anchor_on_lightCone; `:99` decompose	Genuine Mathlib proofs over Complex/Real: anchors have modulus sqrt(2), satisfy Re^2 = Im^2, and z = anchor · coherence holds. Self-contained complex geometry.	I — proof
5	8-fold closure / primitive 8-cycle (gear ratio 3:8).	`DesignAxioms.lean:58` gear_ratio_coprime `(decide)`; `:49` eight_step_phase, `:51` eight_cycle_closes_real `(native_decide)`	gcd(3,8) = 1 is a real Nat proof; phase closure (cos = 1, sin = 0 after 8 steps) is a Float check; the group-theoretic order-8 statement over Complex is not formalized.	II — numeric
6	Coherence C(r) = 2r/(1+r^2), max at r = 1; symmetry C(r) = C(1/r); sech(n·log r) decay.	`Coherence.lean:21` coherence_at_one; `:26` coherence_symmetric_two; `:37` perturbation_sech_sample_r2_n1 `(native_decide)`	Identities verified numerically at sample points r in {0.5, 2, 4}; the universally-quantified theorems (all r > 0) are not proven.	II — numeric
7	mu / coherence govern consensus dynamics (real axis = solve time, imaginary = self-referential target). *(Conjecture)*	No artifact links mu to the executable path. eta = 1/sqrt(2) appears as a difficulty constant (`core/src/dimensional.rs`); the 8-cycle appears only as unproven prediction P2.	Presented as interpretation; no theorem connects the eigenvalue's phase to difficulty, reward, or fork choice.	IV — unbacked
B · WORK SCORE & REWARDS				
8	work_score = log2(solve/verify) · quality, quality in [0,1] (real-valued form). *(Proof-of-Work c)*	`ClassicalAxioms.lean:36` workScoreBitsIdeal_spec `: True`	Placeholder axiom of type True — logically inert; the real-valued work-score formula is not formalized.	III — axiom
9	Doubling solve time adds exactly one bit at quality 1.	`ClassicalAxioms.lean:39` workScoreBitsIdeal_double_solve `: True`	Placeholder axiom of type True — no content.	III — axiom

#	PAPER CLAIM (section)	LEAN EVIDENCE (file : line)	WHAT IS ACTUALLY ESTABLISHED	TIER
10	Deterministic on-chain work score is well-defined (asymmetry gate solve >= 2·verify; quality in basis points).	`WorkScore.lean:80` workScoreFixed; `:48` log2Ratio; `:111` applyQuality_half; `:125` workScoreFixed_ten_to_one	Genuine Nat definitions plus proved lemmas for the integer fixed-point path that actually runs on-chain. Note: integer approximation of log2 (solve/verify = 10 yields 3.25 bits vs true 3.3219).	I — proof
11	Reward = floor(w · S · K / W_parent); more work yields more reward (monotonic). *(Incentive)*	`Rewards.lean:32` mintAtoms; `:77` mintAtoms_mono_work	Genuine Nat proof of weak monotonicity at fixed parent work; matches the paper's emission shape (S = 10^12, K = 50).	I — proof
12	About 50 coins minted on the first block after genesis.	`Rewards.lean:54` first_harvest	Proven: when W_parent = w >= 1, mint = S · K atoms = 50 display units.	I — proof
13	Nonzero asymptotic growth; no hard cap; emission decays as W grows. *(Fee Schedules)*	`Rewards.lean:99` totalMinted (definition only; no asymptotic theorem)	Long-run emission behaviour is defined but not proven; no theorem bounds or characterizes the supply curve.	IV — unbacked
14	Cumulative chain security = sum of per-block bit scores (ideal).	`ClassicalAxioms.lean:42` chainSecurityBitsIdeal_spec : True	Placeholder axiom of type True; the integer aggregate (chainWork) is real — see C3.	III — axiom
C · SECURITY & FORK CHOICE				
15	Attacker success closes exponentially: P approx (q/p)^z — integer core. *(Calculations)*	`SecurityCalculations.lean:38` catch_up_power_lt; `:60` catchup_fortynine_fiftyone_pow_twenty	Proven only that q^z < p^z for q < p (i.e. success probability is strictly below 1). A true but weak statement.	I — proof
16	P approx (q/p)^z as a magnitude bound, via a Poisson work-share race. *(Calculations)*	No probabilistic model formalized; the adversary model is a Boolean structure (`SecurityModel.lean:36`). Paper concedes 'not the decimal cells'.	The (q/p)^z magnitude, the Poisson race, and work-share = security-share are asserted in prose, not proven.	IV — unbacked
17	Heaviest-chain fork choice; positive work strictly increases W; zero-work adds nothing.	`SecurityCalculations.lean:70` chainWork_strict_mono_append; `:75` chainWork_append_zero; `SecurityModel.lean:83` heavierChain_trans	Genuine Nat/Bool lemmas: cumulative work is strictly monotone under appending positive work, invariant under zero-work, and 'heavier' is transitive.	I — proof
18	Adversary cannot forge a passing verify without a witness; honest policy enforced.	`SecurityModel.lean:36` AdversaryCapabilities; `:40` inflate_losing_solve_time := true	Typed documentation of trust assumptions, not theorems. Notably flags that losers can grind solve time (winner sets the competitive time) — the sandbagging vector is acknowledged but unmodeled.	III — axiom
D · PROBLEM COMPLEXITY (Problem Registry)				
19	Subset-Sum is NP-complete.	`ClassicalAxioms.lean:12` subsetSum_NPC : True (also `SubsetSum.lean:12`)	Placeholder axiom; classically correct (Karp 1972) but not formalized.	III — axiom
20	3-SAT is NP-complete (Cook–Levin).	`ClassicalAxioms.lean:18` threeSat_NPC : True (also `Sat3.lean:18`)	Placeholder axiom; classically correct but not formalized.	III — axiom
21	Decision TSP is NP-hard.	`ClassicalAxioms.lean:24` decisionTsp_NPH : True (also `TspFeasibility.lean:17`)	Placeholder axiom; classically correct but not formalized.	III — axiom
22	Factorization is NP-hard ('NP-H' in the Future Problem Types table); verify O(1).	No Lean artifact; not present in the axiom set.	Unbacked and incorrect on the merits: integer factorization is in NP and co-NP and is not known to be NP-hard; verifying p·q = N is not O(1).	IV — unbacked
E · PRIVACY / ZERO-KNOWLEDGE (User Submissions Integration)				

#	PAPER CLAIM (section)	LEAN EVIDENCE (file : line)	WHAT IS ACTUALLY ESTABLISHED	TIER
23	Private submissions use a zero-knowledge proof of well-formedness. *(Private Submissions)*	`SecurityModel.lean:120` placeholderProof_binding; `:123` placeholderProof_sound_testnet; `:126` production_requires_real_zk — all `: True`	The repository implements a testnet SHA-256 MAC placeholder, not ZK; in-code comments state production must use real ZK, which is not present. Described as deployed; is a placeholder.	IV — unbacked
F · FALSIFIABILITY (Conjecture)				
24	Three falsifiable predictions: sech recovery; 8-cycle periodicity; log-ratio symmetry.	`Falsifiability.lean:26` predictedRecovery; `:30/:34/:38` prediction1/2/3; `:47` conjectureSupported	A decidable test harness is defined; the predictions are empirical and intentionally not proven a priori. Correctly scoped — listed for transparency, not as a deficiency.	III — axiom
G · BUILD / REPRODUCIBILITY (Verify Lean Proofs)				
25	Builds with zero errors on Lean v4.14.0 / Mathlib v4.14.0.	`lean-toolchain` = v4.28.0; `lakefile.lean` requires mathlib4 @ v4.28.0	Version mismatch: the stated toolchain does not match the repository; reproduction per the Verify Lean Proofs Independently section as written would fail.	IV — unbacked

Tier shown per row is the verification class of the strongest backing artifact for that claim; where a claim is split across rows (e.g. C1/C2) the split itself is the finding. Line numbers reference commit `005fcf1`.

5 Material discrepancies (correct before circulation)

ITEM	PAPER SAYS	REPOSITORY SHOWS
Zero-knowledge privacy Private Submissions	Private submissions publish ‘a zero-knowledge (ZK) proof of well-formedness’.	A testnet SHA-256 MAC placeholder. `production_requires_real_zk : True` flags that real ZK is future work and not present. **Reads as deployed; is not.**
Toolchain version Verify Lean Proofs	‘Builds with zero errors on Lean 4 v4.14.0 with Mathlib v4.14.0.’	`lean-toolchain` pins v4.28.0 and `lakefile.lean` requires mathlib4 @ v4.28.0. Reproduction as written would fail on the version.
Factorization class Future Problem Types	Listed ‘NP-H’ (NP-hard) with verify cost O(1).	Not in the Lean axiom set. On the merits, factorization is in NP and co-NP and is not known to be NP-hard; verifying p·q = N is not O(1).

6 Recommendations

R1 **Propagate the repo’s own labeling.** Annotate each Lean citation [6]–[9] with its verification class: machine-checked over Nat/Real/Complex, numeric `native_decide` on Float, or axiom cited-not-reproved. The files already carry this; the paper need only mirror it.

R2 **Fix the toolchain line.** The Verify Lean Proofs Independently section should read Lean v4.28.0 / Mathlib v4.28.0.

R3 **Rewrite the Private Submissions paragraph.** Disclose that private-submission well-formedness is currently a testnet SHA-256 MAC placeholder and that production-grade ZK is future work. Do not present ZK as implemented.

R4 **Repair the Problem Registry tables.** Remove or relabel the ‘Factorization NP-H’ entry (factorization is in NP and co-NP, not known NP-hard) and correct the O(1) verify-cost cells for Factorization and SVP.

R5 **Keep [8]; soften the rest of the mu prose.** The `mu_unique` citation is fully earned over the complex field — keep it. But change ‘formalized in DesignAxioms.lean’ (the unitarity / 8-fold / coherence cluster) to ‘numerically checked,’ and add one sentence noting mu’s phase does not enter the executable consensus path (only eta = 1/sqrt(2) and the predicted 8-cycle period do).

R6 **Add a trusted-base note.** Wherever `native_decide` results are cited, state that such checks add the Lean compiler and `Lean.ofReduceBool` to the trusted base and assert floating-point behaviour rather than real-number identities.

R7 **Down-scope or strengthen the security citation.** Either formalize the catch-up bound as a probabilistic / recurrence statement (even discretely), or restrict [9] in the text to ‘monotonicity of cumulative work plus the integer inequality q^z < p^z,’ so it is not read as a proof of the (q/p)^z magnitude bound.

R8 **Optional — raise the rigor instead of hedging.** Port the design-axiom and coherence identities from Float to Real/Complex using the Mathlib machinery already present in `ComplexDecomposition.lean`, and prove the real-valued log2 asymmetry lemmas. That would let the `workScoreBitsIdeal_*` and `chainSecurityBitsIdeal_*` placeholder axioms be deleted and replaced with theorems.

7 Limitations & caveats

- Scope is limited to the whitepaper-cited files and their immediate dependencies in the `lean4/Coinjecture/` tree at the cloned HEAD. The broader 'Eigenverse' proof corpus referenced in in-code comments (`proofs/README.md`) was not in scope.
- The audited tree totals approximately 1,117 lines across 18 files — this is the consensus-specific subset, not the full project formalization.
- Classification was performed by source inspection and a hole sweep (`sorry` / `admit` / `axiom` / `native_decide`); the Lean kernel was not re-executed and a full Mathlib build was not run. Tier-I designations mean 'uses legitimate kernel-checkable tactics and carries full logical content,' contingent on the stated zero-error build.
- `native_decide` proofs are lower-assurance than kernel reductions and depend on compiler correctness; Float equalities are statements about IEEE-754 rounding, not real-number identities. Several design-axiom checks pass because the relevant value rounds to an exactly-representable Float (e.g. (1/sqrt(2))^2 rounding to 0.5), which is a numerical fact, not an analytic one.
- An `axiom X : True` declaration is logically inert: it neither constrains the model nor can serve as a non-trivial hypothesis downstream. Such declarations function as named placeholders, and their presence is neither dishonest nor load-bearing — the files label them as Tier-B.

Appendix A File inventory

FILE (.lean)	LOC	ROLE	BACKING
`ComplexDecomposition`	221	Eigenvalue mu, anchors, Euclidean/Lorentzian charts.	Complex/Real proofs (Mathlib)
`WorkScore`	137	Fixed-point integer work score (on-chain path).	Nat proofs
`SecurityModel`	128	Trust model, fork choice; ZK privacy placeholders.	Nat/Bool lemmas + axioms (: True)
`Rewards`	110	On-chain reward mechanics (emission formula).	Nat proofs
`SecurityCalculations`	79	Catch-up inequality + chain-work monotonicity.	Nat proofs
`DesignAxioms`	65	Design constants (eta, mu, 8-cycle).	Float native_decide
`Coherence`	59	Coherence C(r) and sech identities.	Float native_decide (samples)
`Falsifiability`	54	Empirical-prediction test harness.	defs (harness)
`ClassicalAxioms`	44	NP results + ideal work-score interpretation.	axioms (: True)
`Fixtures`	44	Numeric regression fixtures.	Float native_decide
`DimensionalPools`	42	Dimensional pool scaling checks.	Float native_decide †
`Verify`	33	Polynomial-time solution checkers.	lemmas (no sorry) †
`GeneratorInvariants`	28	Problem-generator invariants.	no holes flagged †
`TspFeasibility`	22	TSP feasibility type + NP-hardness axiom.	axiom (: True) †
`Sat3`	20	3-SAT type + NP-completeness axiom.	axiom (: True) †
`Complexity`	17	Complexity-class scaffolding.	no holes flagged †
`SubsetSum`	14	Subset-sum type + NP-completeness axiom.	axiom (: True) †
`Coinjecture (root)`	—	Root module; imports submodules.	aggregator †

† Glimpsed via the hole sweep but not individually read in full; backing noted is from grep, not line-by-line review.

Appendix B Reproduction

```
# clone only the Lean tree, audited at commit 005fcf1 (2026-05-30)
git clone --depth 1 --filter=blob:none --sparse \
    https://github.com/COINjecture-Network/COINjecture2.0.git
cd COINjecture2.0 && git sparse-checkout set lean4

# hole sweep used for tier classification
grep -rnE '\bsorry\b|\badmit\b|axiom .*: *True|native_decide' \
    lean4/Coinjecture/*.lean

# confirm toolchain (expect v4.28.0, not the paper's v4.14.0)
cat lean4/lean-toolchain ; grep mathlib lean4/lakefile.lean

# full kernel re-check  (heavy; NOT run in this audit)
cd lean4 && lake exe cache get && lake build
```

Prepared by DARQ Labs LLC as an internal pre-publication review of COINjecture Network LLC's whitepaper v2.6. Findings are scoped to the artifacts and commit named above and reflect source inspection, not an independent kernel re-execution. `DARQ-LV-001 v0.2.0` — identifier provisional, pending DARQ document-control assignment.